

UROP Report: Real-Robot Reinforcement Learning with World Models and Vision-Language-Action Models (Progress Report)

Student Name: WANG, Zhixin

Department : CSE

Supervisor: GUO, Song

May 4, 2026

Abstract

This report summarizes progress on an UROP project investigating whether modern video generation systems can serve as scalable world models that capture physical laws from visual observations. We focus on translating Image into simulation-ready 3D assets and evaluating generalization under in-distribution, combinatorial, and out-of-distribution settings. Building on an existing pipeline that combines vision-language prompting, 2D-to-3D decoding, mesh post-processing, and URDF generation, we reproduce baseline results, diagnose practical failure modes (e.g., GPU memory spikes, mask-driven evaluation artifacts, and reconstruction ambiguities), and propose targeted fixes. Preliminary findings suggest strong imitation within training-like conditions but limited abstraction of underlying physical rules.

Keywords: world models; video generation; physics; real-to-sim; 3D reconstruction; evaluation

1 Introduction

Real-robot reinforcement learning requires a large amount of data. However, collecting such data in the real world is often difficult, time-consuming, and costly. **Therefore, finding reliable and**

scalable ways to generate data is important.

In recent months, the Large Language Models (LLMs) have developed rapidly. Many researchers also notice that the coding and vision capabilities of LLMs can be leveraged to generate 2D or 3D assets on a scale, such as PhysX-Anything [9] and ROBOTARENA ∞ [11]. In addition, there are several attempts to use the world model to produce videos that adhere to the law of physics, such as Hunyuan-GameCraft-2 [16], PhysTwin [12], and Vid2Sim [10].

However, these works still have some drawbacks. As mentioned in “How Far is Video Generation from World Model” [13], current world model or video generation models **fail to abstract general physical rules** and instead exhibit “case-based” generalization behavior. In addition, when generalizing to new cases, models are better for predicting the color instead of more important physical quantity, such as velocity and shape. So, to solve this problem, we consider an alternative perspective: **“let the machine learn the physical law might be hard, but use existing tools, such as some reliable physical engine [4, 6, 5], and cooperate with Large Language Model’s (LLM) [1, 7, 3] strong coding skill, producing videos that adhere to the physical law might be possible”**.

Motivated by this idea, we aim to develop a pipeline that produces short videos that adhere to the physical law while maintaining high visual quality. Based on existing work, MoReGen [8], we develop a modular, multi-agent coding and rendering framework that transforms an input image and a natural-language motion description into executable Newtonian simulations, enabling scalable and physically grounded video generation.

2 Related Work

We review recent research on *text-to-video* (T2V) generation models, Video2Sim pipelines, and the evaluation method, focusing on the reason why they might produce videos that are against the law of physics. In addition, we examine how previous work assesses physical adherence and physical commonsense in generated videos.

2.1 Video generation and generalization

As reported by Bingyi Kang et al. [13], what the models learn might not be the physical law. The models behave more like case-by-case studies, and they cannot show the ability to generalize behavior. For example, in the **In-distribution** (ID), the error is 0.012, but in the **Out-of-distribution** (OOD), the error is 0.427, which is around 40 times higher, suggesting that the current T2V models struggle to display the ability to generalize behavior.

2.2 Video-to-simulation pipelines

To address this problem, Xiangyu Bai et al. propose a new method [8]. They introduce a new stable pipeline, called **MoReGen**, to conquer the problem of OOD. In their MoReGen, they try to use the strong coding ability of current LLM, such as GPT-5 [1], Qwen [7], and Kimi-2.5 [3].

Their pipeline has 3 stages. **Stage 1: Text Parsing.** They use **Text Agent**, a fine-tuned Qwen2.5-Coder-14B [7], to rephrase user input and generate a complete and physically coherent description. **Stage 2: Code Synthesis.** The rephrased prompt is fed into the **Coder Agent**, which produces the corresponding code. The code is executed and rendered using a physical engine, such as manim [4] or pymunk [6]. **Stage 3: Visual Feedback.** a **Vis-Agent**, Qwen [7], would be responsible for this part. The agent will look at the rendered video and describe what he sees in the video and whether the output video reaches the original intent.

Although MoReGen can generate videos that adhere to the law of physics, the equality of video it produces is worse than some of T2V models (see Figure 1 and Figure 4(a), 4(b)). Moreover, despite MoReGen overcoming the OOD problem by introducing a physical engine, which can produce videos that follow the law of physics, the performance of reconstructing complex objects is poor (see Figure 2), making the final result unreal. Additionally, the input of MoReGen is limited to text, since it relies on an agent to describe the scene precisely in text.

T2V Models	Trajan [1]		VideoPhy2 [6]	
	AJ $\uparrow$	OA $\uparrow$	SA $\uparrow$	PC $\uparrow$
Wan2.2	0.38 ± 0.02	0.94 ± 0.02	2.88 ± 0.46	3.77 ± 0.68
LTXV	0.79 ± 0.03	0.99 ± 0.02	2.95 ± 0.28	3.75 ± 0.68
CogVideoX	0.17 ± 0.02	0.72 ± 0.04	2.83 ± 0.47	4.07 ± 0.59
Veo3	0.15 ± 0.02	0.82 ± 0.02	2.95 ± 0.33	3.49 ± 0.50
Grok	0.11 ± 0.02	0.72 ± 0.05	3.04 ± 0.34	3.76 ± 0.56
Sora2	0.45 ± 0.02	0.92 ± 0.02	3.01 ± 0.35	3.91 ± 0.52
NewtonGen	0.56 ± 0.02	0.96 ± 0.02	2.77 ± 0.42	4.21 ± 0.52
WISA	0.41 ± 0.02	0.86 ± 0.03	2.92 ± 0.45	4.01 ± 0.66
MoReGen (Ours)	0.10 ± 0.03	0.64 ± 0.05	2.73 ± 0.44	4.53 ± 0.69

Figure 1: performance of MoReGen pipeline.

3 Project Goals and Problem Definition

Previous work suggests that current T2V models cannot achieve visual fidelity and physical plausibility at the same time. For example, the traditional diffusion model performs poorly in physical compliance, and MoReGen performs poorly in visual fidelity. This project seeks to mitigate this trade-off by integrating complementary strengths from diffusion-based generation and physics-engine-based simulation.

Accordingly, our program goal is to **build up an Image-to-Video (I2V) workflow, which can automatically generate video based on the first frame and a short prompt describing the objects’ motion.**

4 System Overview (Pipeline stages)

4.1 Stage 1: Generating the object mesh

Since MoReGen cannot reconstruct complicated objects within the physical engine, we introduce a new method: reconstruct objects using mesh-based assets in the simulator, allowing the coder agent to focus on modeling object motion.

Several physical engines, such as pybullet [5] support representing objects using mesh/URDP files. However, manually creating meshes on a large scale is not feasible. Therefore, this work seeks a scalable and stable approach for generating object meshes with good visual quality. To this end, we find PhysX-Anything [9], a pipeline that can produce high-quality meshes (see Figure 5)

4.2 Stage 2: Coding and Simulation

A mesh only describes the shape of the object. To produce motion in simulation, the object must be subjected to external forces that cause it to move, rotate, or deform. That needs an agent to do the coding work. Thus, we adopt the MoReGen’s pipeline [8] (using their Coder Agent to produce the motion, and using the Vis-Agent to give the feedback and guide the Coder Agent to do the adjustment). The output will be a coarse video that depicts the object’s motion (see Figure 3).

4.3 Stage 3: Rendering

In this stage, we use cosmos-transfer2.5-2B [2] to render the semi-finished video. The model supports multiple conditioning inputs, including text prompt, noise video, depth video, edge maps and camera information. It then produces a visually refined video with improved visual quality (see Figure 4(c))

5 Experiments and Progress

5.1 Reproducing baseline results

The initial experiments focused on reproducing the results reported for MoReGen. Using the pipeline provided by Xiangyu Bai et al. [8], the Coder Agent (Qwen 2.5) was unable to produce **runnable** programs. To improve reliability, we substituted the Coder Agent with more capable LLMs, including GPT5.2 [1] and Kimi-2.5 [3]. Figure 4(a) shows the result produced by GPT-5.2 and Figure 4(b) shows the result produced by Kimi-2.5.

We also evaluated alternative physics engines. Xiangyu Bai et al. choose manim [4] and pymunk [6]. However, the render outcome is limited and pymunk does not support Linux environment. Consequently, we switched the physical engine to pybullet [5], which supports mesh-based inputs and offers broader platform compatibility. Moreover, this change improved the quality of the generated videos. (Figure 2 shows the video frames generated by manim-based agent; Figure 3 shows the video frames generated by pybullet.)

In addition, we use PhysX-Ahything [9] to generate meshes. It uses a pretrained model (based

on Qwen2.5) to produce the geometry information and then applies TRELLIS [17] to produce visible mesh. We successfully replicated the given demo (see Figure 5). However, generalization to other objects, such as shoes, remains limited (see Figure 6).

5.2 Evaluation setup and metrics

Given outcome video frames, evaluating the result correctly and consistently is important for our further improvement. Since our project focuses on **physical adherence**, we adopt the evaluation protocol mentioned in PISA Experiments [14] as our evaluation method.

In the PISA Experiments, their evaluation metric focused on the following three dimensions: **Trajectory error** (L2 loss), **Chamfer Distance** (CD) and **Intersection over Union** (IoU). The Trajectory error measures motion consistency, Chamfer Distance reflects the shape fidelity, and IoU shows the degree of agreement between the target videos and outputs.

5.3 engineering challenge and debug

During the experiments, several practical challenges were observed. First, when generating the meshes, CUDA frequently reported out-of-memory (OOM) errors. We attempted to use different graphic cards with more memory space (48GB), but the issue persisted. Therefore, We tried to offload the TRELLIS model [17] to the CPU, which solved the OOM at the cost of running time. We plan to continue investigating the issue and try to solve it in 2-4 weeks.

Second, in the PISA experiment, their evaluation code lacks an important part, points, indicating which objects need to be followed. We manually added this information, after which the evaluation pipeline ran correctly. To reduce manual effort, We used the grounding-dino-base model [15] to locate the target object, enabling more efficient generation of the required labels.

6 Current Results (Preliminary)

6.1 Qualitative results

Comparing Figure 2 and Figure 3, one can clearly notice that our pipeline produces more real videos, even without the rendering process in stage 3. That suggests that switching the Coder

Agent and the physical engine used is an effective way to produce more realistic video. And after rendering, Figure 4(c) has much better visual performance than Figure 4(a), which is more plausible.



Figure 2: Prompt: A shoe fall. Video generate using manim as physical engine.

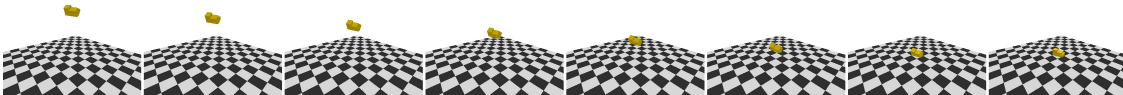


Figure 3: Prompt: A shoe fall. Video generate using pybullet as physical engine.

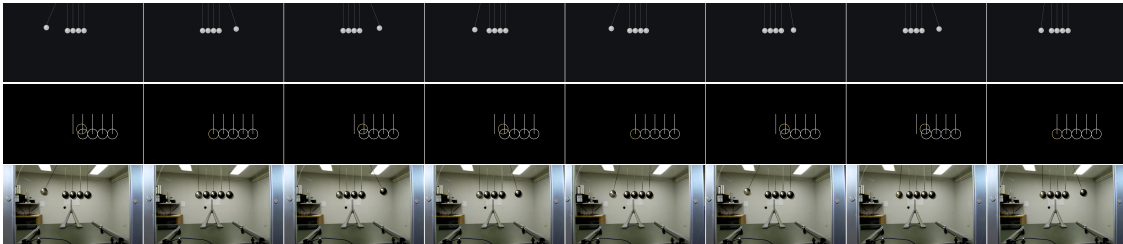


Figure 4: Image from top to down is Figure 4(a), 4(b), 4(c) respectively. And the prompt is from MoReGen’s paper directly. Video generate using manim as physical engine.

6.2 Quantitative results

As shown in Table 1, we compare the masks generated in our pipeline with the masks provided in the PISA experiments. The ”standard” row reflects the loss when two masks are identical. The ”samples loss” row represents the loss between my masks for that sample and the given mask.

Table 1: Evaluation metrics on selected samples.

Sample	L2 ↓	CD ↓	IoU ↑
standard	0.00	0.00	1.00
seen 01	0.00045	9.6230e-05	0.92399
seen 28	0.00036	6.5913e-05	0.92901
seen 29	0.00030	3.3823e-05	0.96526

Table 2 shows the improvement brought by switching the physical engine from manim to pybullet. In the experiment, the agent is asked to create a video in which a shoe falls onto the ground. Clearly, in L2 loss and Chamfer Distance, our pipeline performs better.

Table 2: Evaluation metrics on selected samples.

Physical engine	L2 ↓	CD ↓	IoU ↑
manim	0.19989	0.53647	0.00
pybullet	0.14223	0.35443	0.00

6.3 Limitation

Although our pipeline can generate videos that follow physical laws and appear reasonably realistic, the current approach still has several limitations.

First, PhysX-Anything [9] is unable to produce meshes for general objects, such as shoes (see Figure 6). It performs well in the given demo, creating a high-resolution mesh of a faucet. However, while attempting to generate a mesh of a shoe, it fails, regardless of the input image resolution. To find the reason, we read the log and found that the pre-trained model cannot recognize the object category. For example, it intends to classify a shoe as furniture. As Ziang Cao et al. [9] note, the pretrained model can be further trained on task-specific datasets, so additional training may be a key point.



Figure 5: Given demo in PhysX-Anything. The left handside is the input image and right handside is the result.



Figure 6: Mesh created by PhysX-Anything. The left handside is the input image and right handside is the result.

Second, Cosmos [2] may mistakenly treat the shadow of the object as a part of the object. In Figure 4(c), there are several dark regions under the pendulums, which is the shadow. To solve this problem, my collaborator, Xinyi HAN, suggests that this issue may be mitigated by applying stricter edge conditioning; she is currently working on this direction.

7 Future Work

This project is still in progress. In the following 4-6 weeks, I will:

1. Dive into PhysX-Anything, find a way to facilitate the mesh quality. Maybe the given pre-train model needs training process, It might need a training process.
2. Reduce the memory consumption and running time of decoder.

3. Coordinate with my partner, Xinyi HAN, Find a solution to the shadow and drifting problem

8 Conclusion

In this project, our aim is to build an I2V pipeline that can create realistic video that conforms to physical laws, conditioned on an image and a motion description (text). Although this project is not yet complete, I still learned a lot from this project, including PyTorch usage, multiple ways to represent objects, and using existing model to handle complex tasks (coding, locating object in the picture). The next phase of work will be carried out in 4–6 weeks.

References

- [1] Gpt series large language models (referenced as gpt-5 in this report). <https://openai.com/zh-Hant-HK/index/introducing-gpt-5-2/>.
- [2] cosmos-transfer2.5-2b. <https://github.com/nvidia-cosmos/cosmos-transfer2.5>, 2026. Accessed 2026-05-09.
- [3] Kimi 2.5. <https://www.moonshot.cn/>, 2026. Accessed 2026-05-09.
- [4] Manim: Mathematical animation engine. <https://www.manim.community/>, 2026. Accessed 2026-05-09.
- [5] Pybullet: Python module for physics simulation. <https://pybullet.org/wordpress/>, 2026. Accessed 2026-05-09.
- [6] Pymunk: Pythonic 2d physics library. <https://www.pymunk.org/>, 2026. Accessed 2026-05-09.
- [7] Qwen2.5-coder. <https://huggingface.co/Qwen>, 2026. Accessed 2026-05-09.
- [8] Xiangyu Bai, He Liang, Bishoy Galoaa, Utsav Nandi, Shayda Moezzi, Yuhang He, and Sarah Ostadabbas. Moregen: Multi-agent motion-reasoning engine for code-based text-to-video synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [9] Ziang Cao, Fangzhou Hong, Zhaoxi Chen, Liang Pan, and Ziwei Liu. Physx-anything: Simulation-ready physical 3d assets from single image. *arXiv preprint arXiv:2511.13648*, 2025.
- [10] Chuhao Chen, Zhiyang Dou, Chen Wang, Yiming Huang, Anjun Chen, Qiao Feng, Jiatao Gu, and Lingjie Liu. Vid2sim: Generalizable, video-based reconstruction of appearance,

- geometry and physics for mesh-free simulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26545–26555, 2025.
- [11] Yash Jangir, Yidi Zhang, Pang-Chi Lo, Kashu Yamazaki, Chenyu Zhang, Kuan-Hsun Tu, Tsung-Wei Ke, Lei Ke, Yonatan Bisk, and Katerina Fragkiadaki. Robotarena ∞ : Scalable robot benchmarking via real-to-sim translation, 2025. URL <https://arxiv.org/abs/2510.23571>.
- [12] Hanxiao Jiang, Hao-Yu Hsu, Kaifeng Zhang, Hsin-Ni Yu, Shenlong Wang, and Yunzhu Li. Phystwin: Physics-informed reconstruction and simulation of deformable objects from videos. *ICCV*, 2025.
- [13] Bingyi Kang, Yang Yue, Rui Lu, Zhijie Lin, Yang Zhao, Kaixin Wang, Huang Gao, and Jiashi Feng. How far is video generation from world model? – a physical law perspective. *arXiv preprint arXiv:2406.16860*, 2024.
- [14] Chenyu Li, Oscar Michel, Xichen Pan, Sainan Liu, Mike Roberts, and Saining Xie. Pisa experiments: Exploring physics post-training for video diffusion models by watching stuff drop. *arXiv preprint arXiv:2503.09595*, 2025.
- [15] Zuwei Long and Wei Li. Open grounding dino: The third-party implementation of the paper grounding dino. <https://github.com/longzw1997/Open-GroundingDino>, 2023.
- [16] Junshu Tang, Jiacheng Liu, Jiaqi Li, Longhuang Wu, Haoyu Yang, Penghao Zhao, Siruis Gong, Xiang Yuan, Shuai Shao, and Qinglin Lu. Hunyuan-gamecraft-2: Instruction-following interactive game world model, 2025. URL <https://arxiv.org/abs/2511.23429>.
- [17] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation. *arXiv preprint arXiv:2412.01506*, 2024.